Modular Personal Productivity Workspace

Student Name: Brayden Bowen
Course: CS 408 - Full Stack Web Development
Semester: Fall 2026

Overview

The app I intend to build will be a full-stack personal management web application intended to combine several productivity and personal tracking tools into a customizable workspace. Users will be able to manage notes, track workouts and fitness progress, monitor personal finances, set goals, and view information through a centralized dashboard. Nexus is a tentative name for the application and is subject to change.

The main concept is inspired by other applications such as Notion and Obsidian, but won't attempt to completely recreate either applications. Instead, it will focus more on personal management by providing specialized modules that share information through a common dashboard.

The application will be developed using Ruby on Rails as the primary framework. PostgreSQL will be used for relational data storage and the application will be hosted on an AWS EC2 instance. There may be additional AWS services such as S3 for storing uploaded files / images.

The two applications mentioned above:

- Notion — https://www.notion.com/
- Obsidian — https://obsidian.md/

Target audience

The target audience for this application will be students, young professionals, and individuals who want to organize several areas of their lives within one location.

Lots of people will use multiple applications for different purposes. An example would be someone using one application for notes, another for tracking workouts, a different spreadsheet for tracking finances, and so on. This application is intended to provide all of these tools in one centralized location.

The application will be especially useful for users who want to track and visualize information about themselves over time. Examples of this include:

- Organizing notes, personal goals, and finances
- Tracking workouts and strength progression
- Learning to budget and monitor spending

Functionality

Users will create an account and log into a private workspace where they will be able to access various modules through a navigation menu.

- **User Accounts:** Users will be able to register, log in/out, and edit basic account and profile information.
- **Dashboard:** The dashboard will act as the homepage of the application after a successful login. It will display summaries from different parts of the application, such as recent notes or workouts, fitness statistics, and monthly spending.
- **Notes:** Will provide a central location for users to store written information. Will allow users to create and edit notes, delete notes, organize them into folders/categories, add tags and search for notes.
- **Fitness Tracker:** Will allow users to record their workouts and track performance over time.
- **Finance Dashboard:** Users will be able to manually track finances by creating financial accounts, entering balances and recording income and expenses. They will also be able to categorize transactions and view them by category.
- **Goals:** Users will be able to create measurable personal goals associated with categories including fitness, financial, and academic goals.
- **Search:** The application will include a global search feature that allows users to search through information in their workspace, particularly in their notes.
- **File Uploads:** The application will also allow users to attach files or images to content. An example of this would be including images with notes or adding a profile picture.

Database Schema

The application will use a relational database with PostgreSQL as the planned database management system. The initial high-level structure will include the following tables:

Users: Stores information associated with application accounts

- **users:** id, email, password_digest/encrypted_password, username, created_at, updated_at

Notes:

- **notes:** id, user_id, title, content, folder_id, created_at, updated_at
- **folders:** id, user_id, name, parent_id, created_at, updated_at

Fitness:

- **exercises:** id, user_id, name, muscle_group, description
- **workouts:** id, user_id, name, workout_date, notes, created_at, updated_at
- **workout_exercises**: id, workout_id, exercise_id, exercise_order
- **exercise_sets:** id, workout_exercise_id, set_number, repetitions, weight

Finances:

- **financial_accounts**: id, user_id, name, account_type, current_balance, created_at, updated_at
- **transactions:** id, user_id, financial_account_id, category_id, description, amount, transaction_type, transaction_date, created_at, updated_at
- **transaction_categories:** id, user_id, name

Goals:

- **goals:** id, user_id, title, description, category, starting_value, current_value, target_value, unit, deadline, completed, created_at, updated_at

Dashboard:

- **dashboard_widgets:** id, user_id, widget_type, position, width, height, settings, created_at, updated_at

Almost all personal information belongs to a specific user so the User model will be the central model in the database

Media

The application will primarily rely on interface elements, icons, charts, and user-generated content.

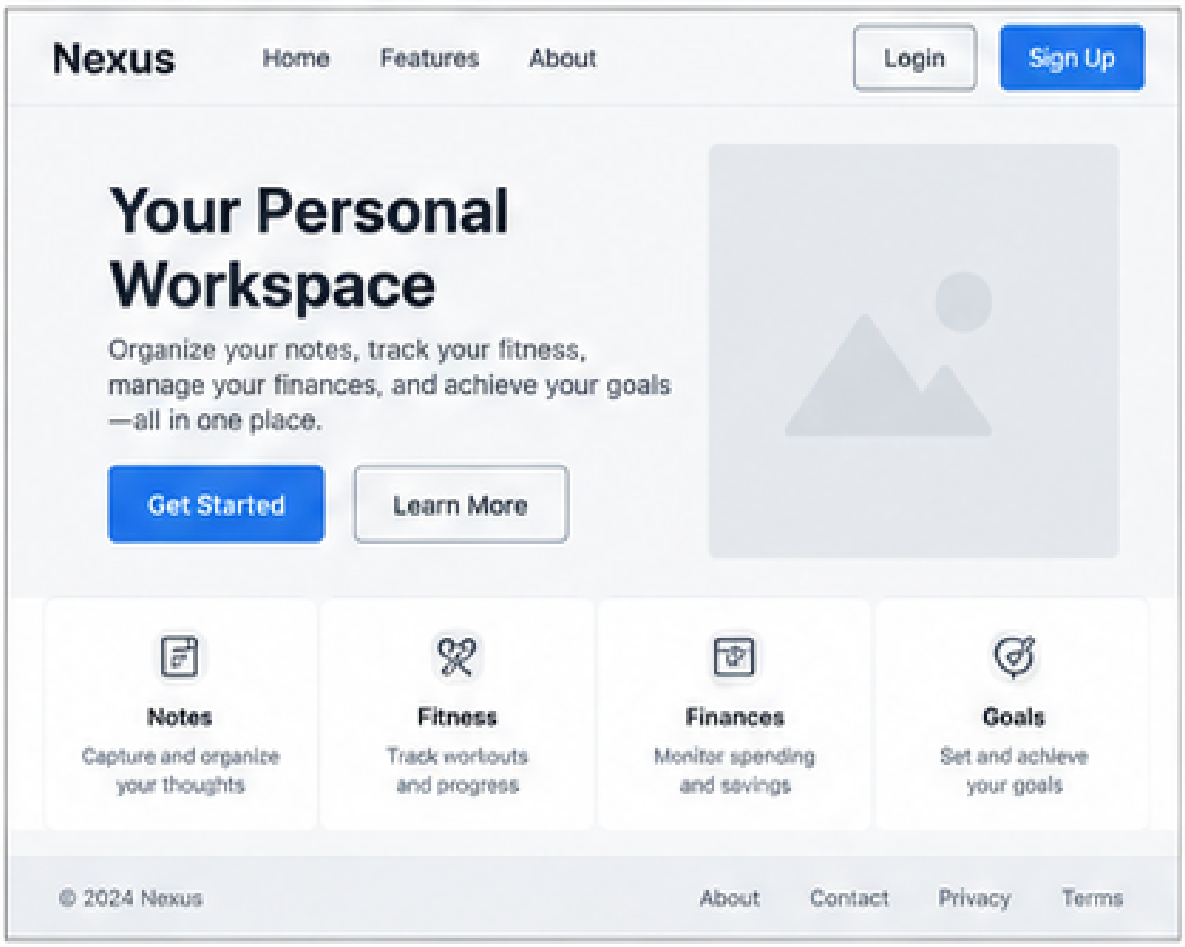
Icons will be used throughout the interface for the navigation and dashboard elements and I plan on using an open-source icon library like Bootstrap Icons or Font Awesome for them.

Charts will be generated using data stored in the application and will represent various things including: exercise strength progression, monthly spending, goal progress, etc.. I will use a JavaScript charting library to generate these.

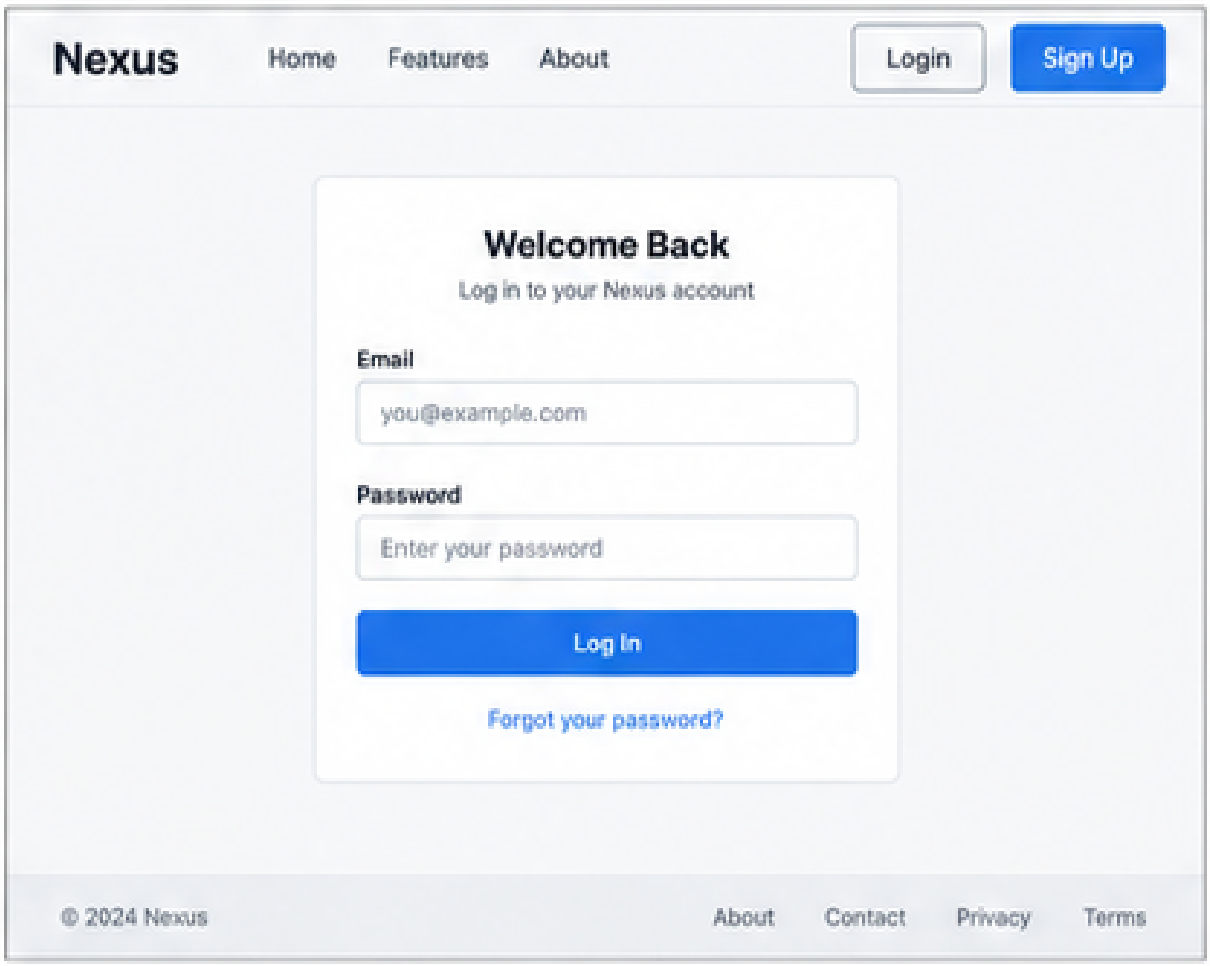
Any images needed will be stock photography or AI-generated and most images (if there are any) will be user-uploaded and stored using Rails Active Storage.

Wireframes

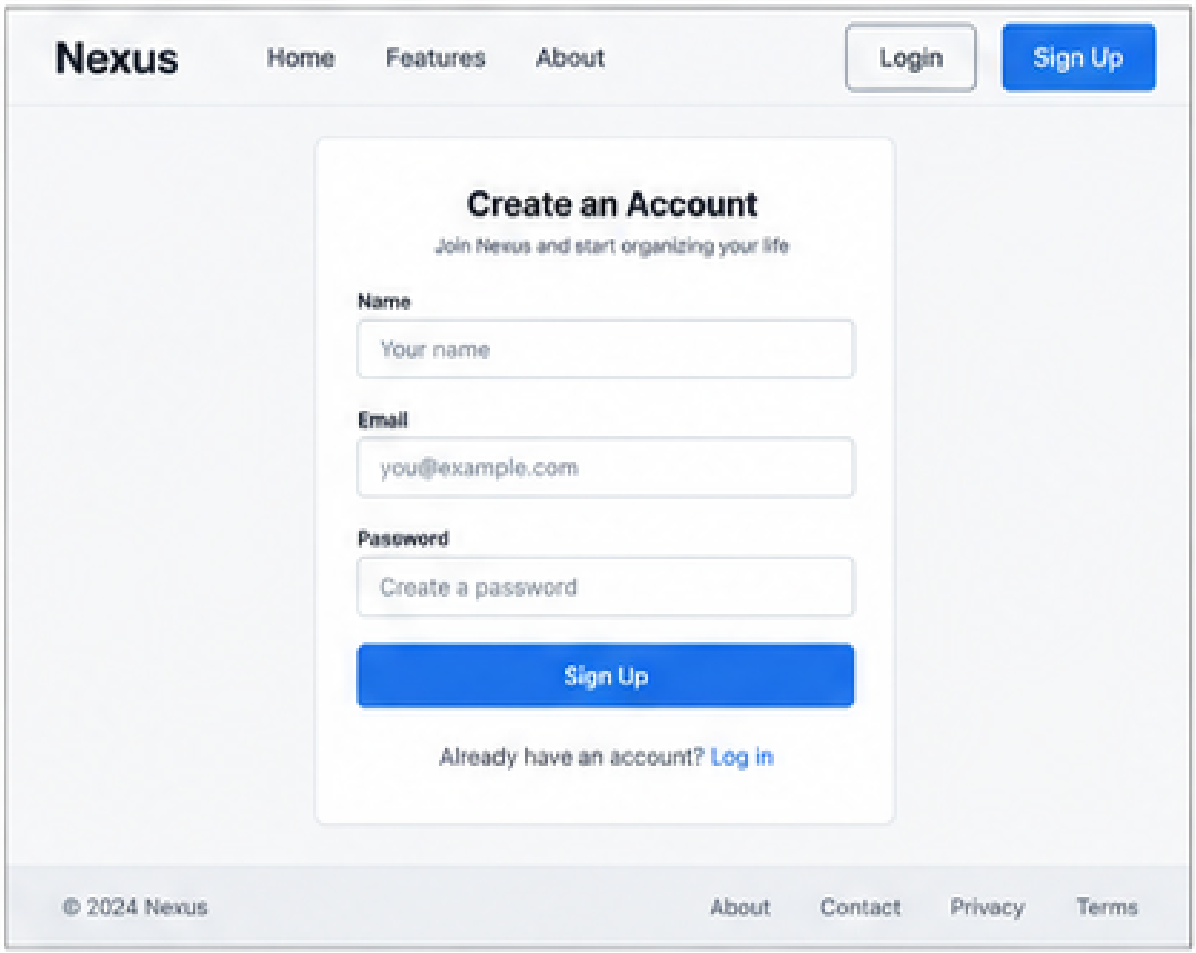
1. Landing Page (Public)



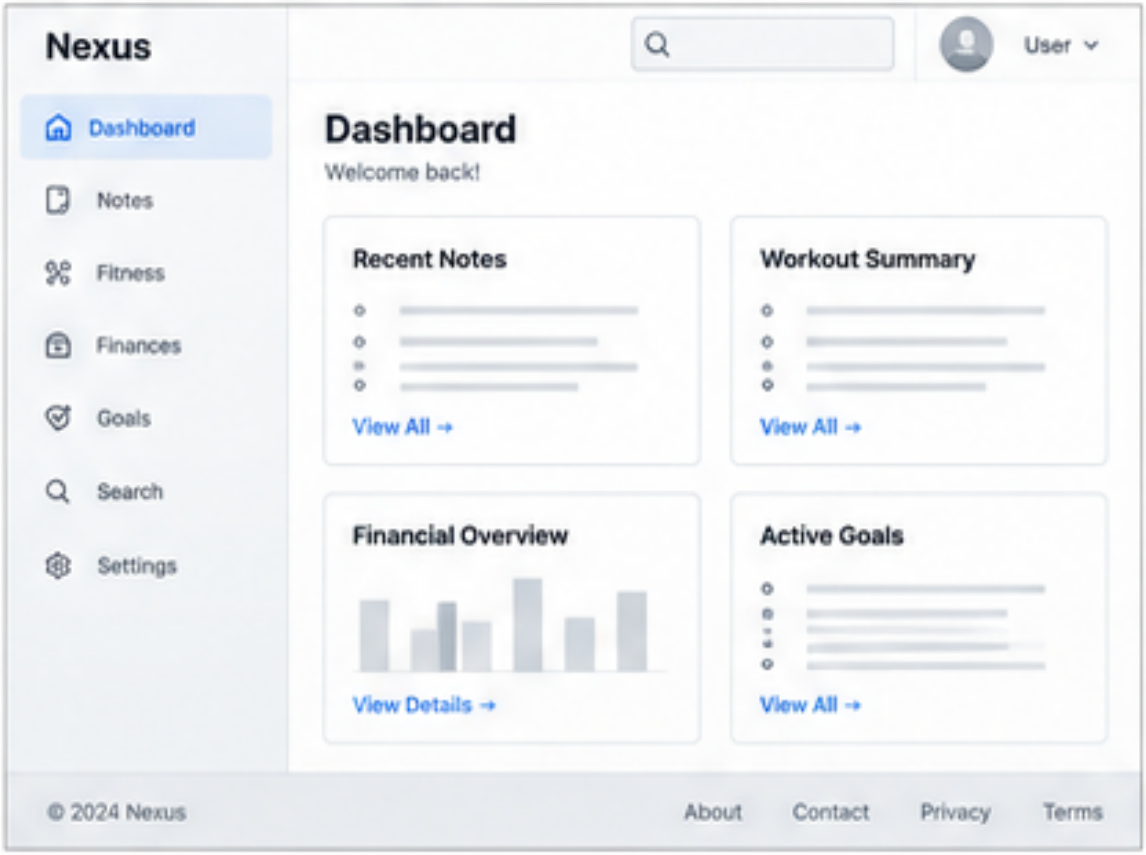
2. Login Page (Public)



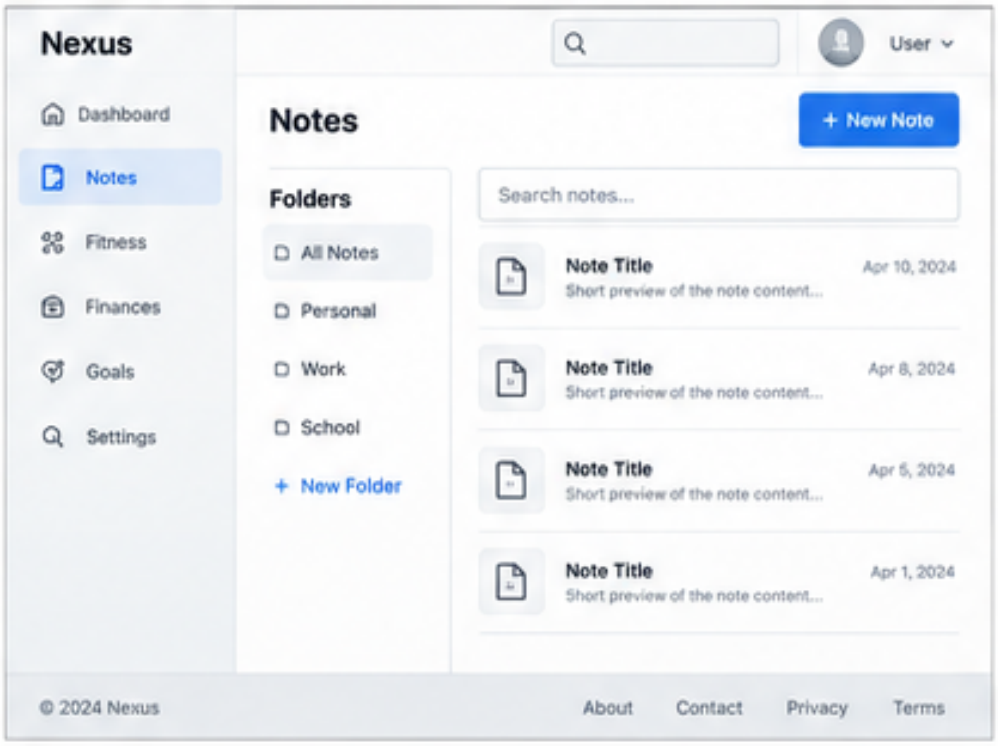
3. Registration Page (Public)



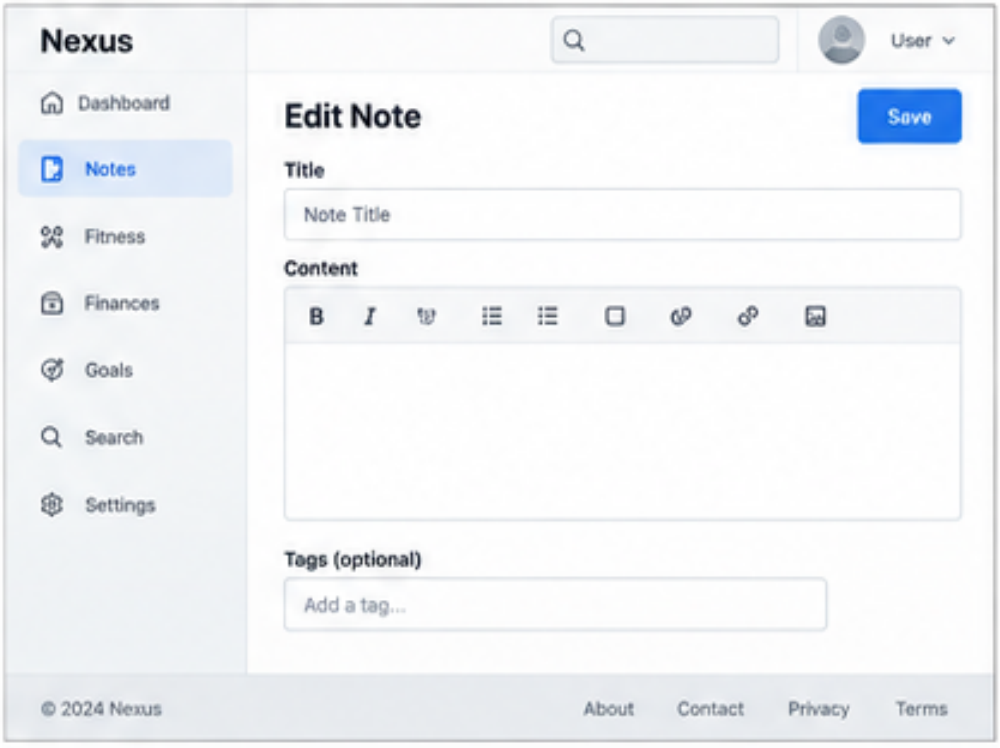
4. Dashboard (Logged In)



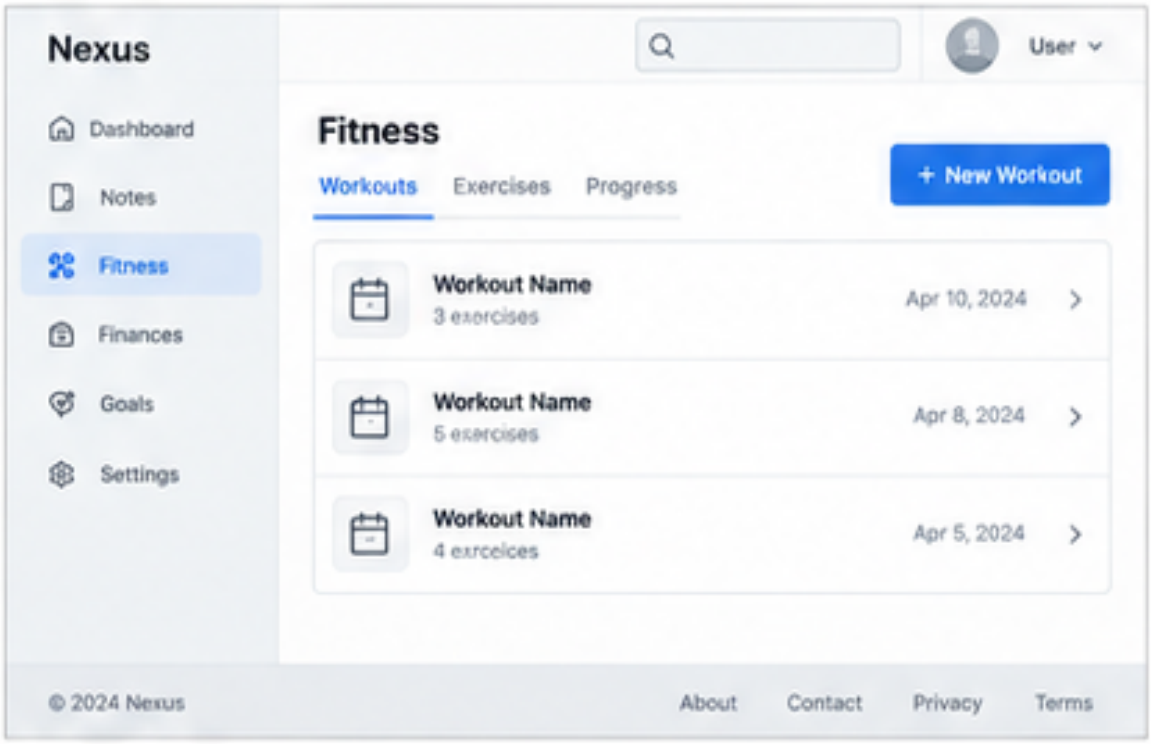
5. Notes Page



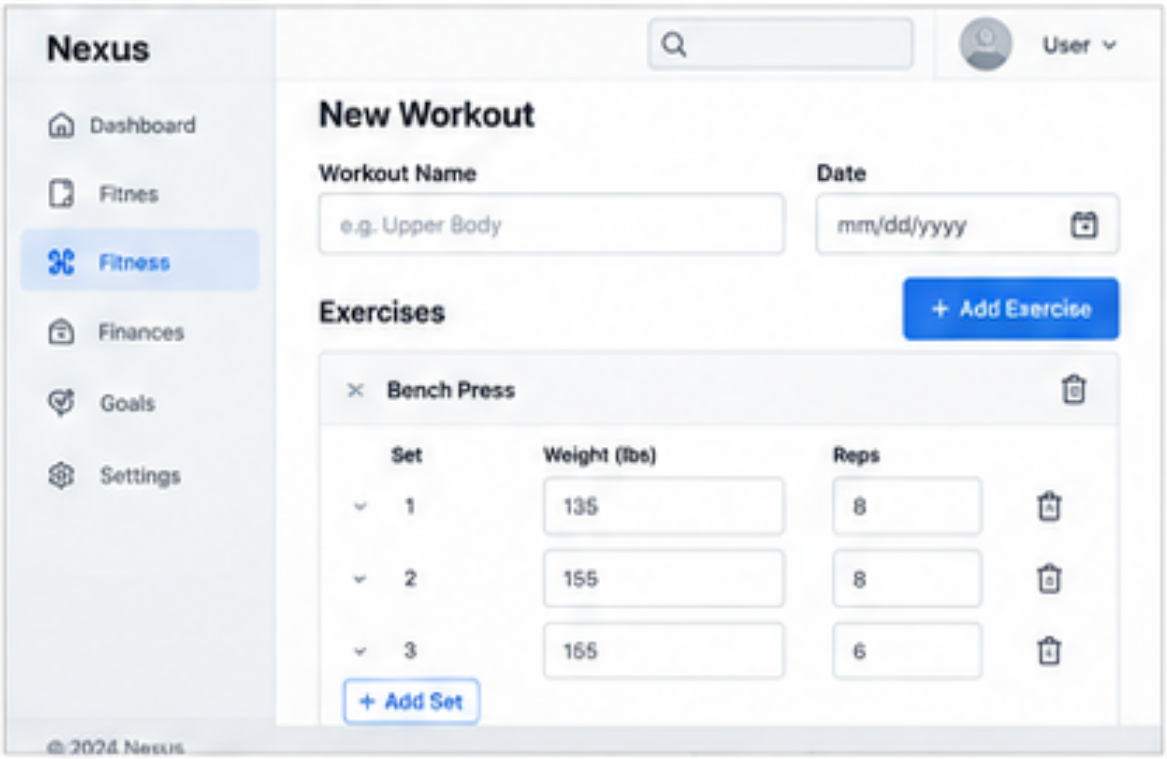
6. Note Editor Page



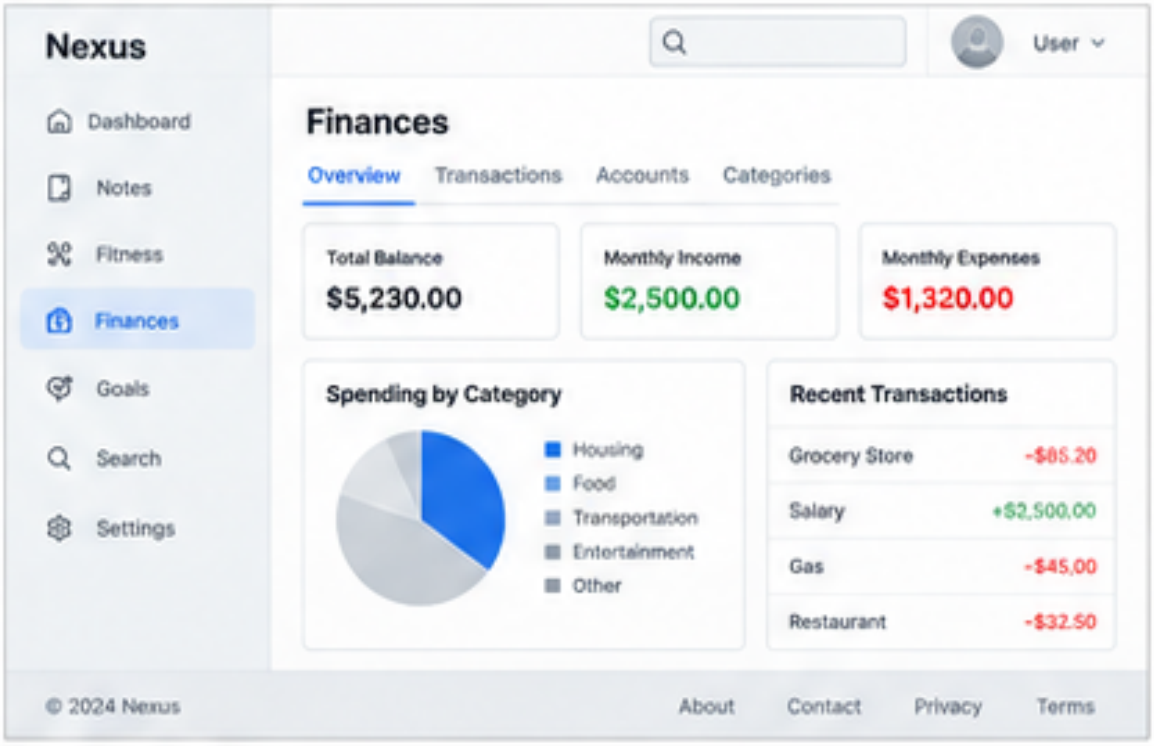
7. Fitness Page



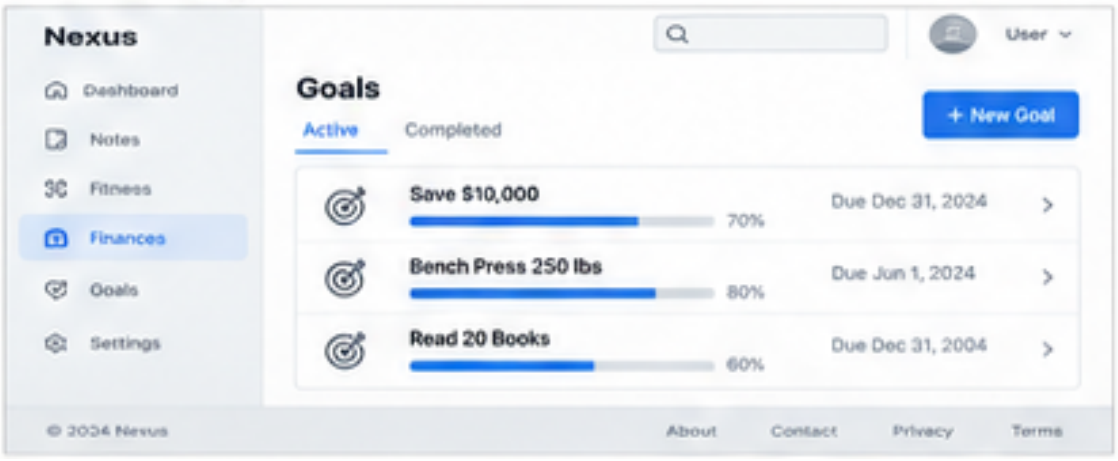
8. Workout Editor Page



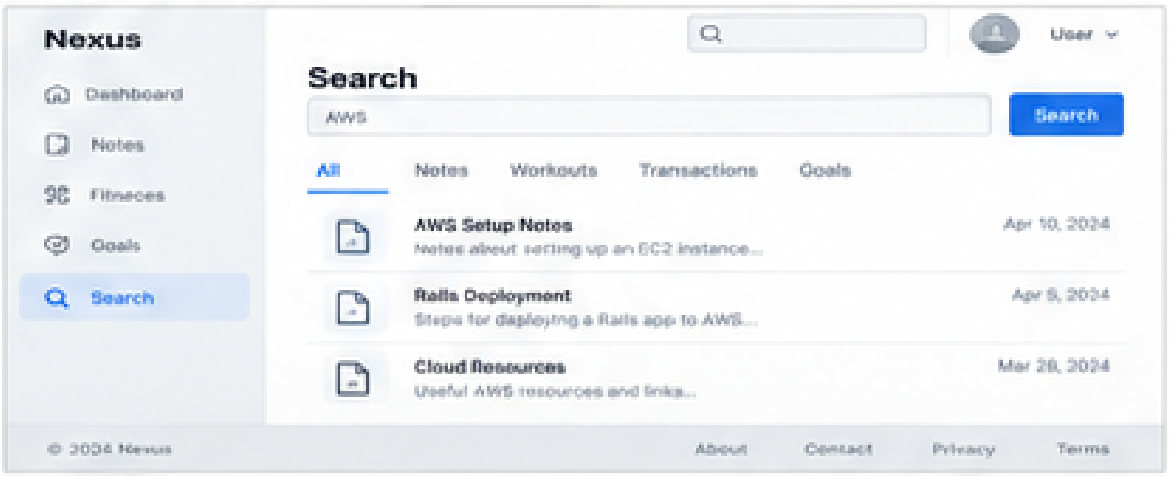
9. Finance Page



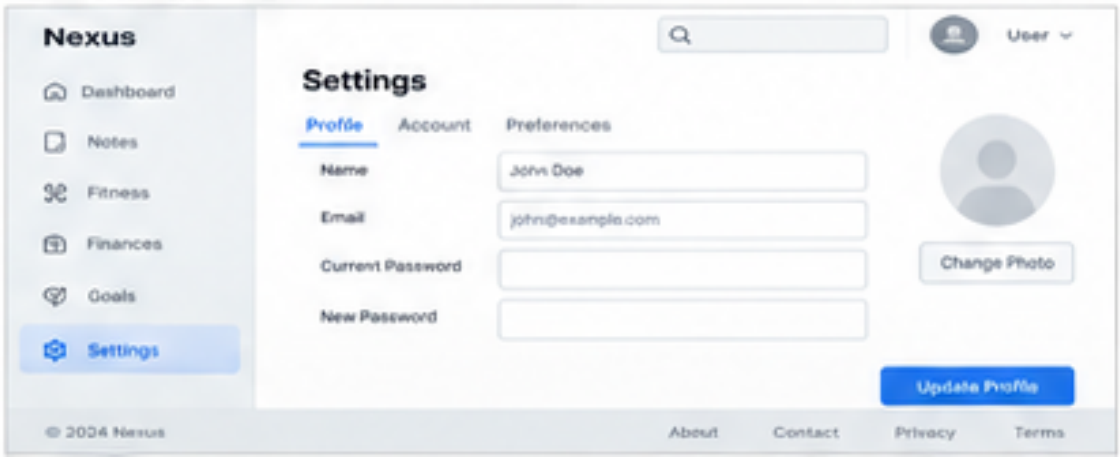
10. Goals Page



11. Search Page



12. Settings Page



Google drive link (if picture too small/blurry):
https://drive.google.com/file/d/1EBtVrkAH6Iamu4iqnbCa8mrDurudp449/view?usp=sharing

Schedule

Checkpoint 1 — Foundation

- The project will be created and PostgreSQL and Bootstrap will be configured and working
- Shred Rails layout, header, and footer created
- Landing page completed with main navigation implemented
- User registration and login/logout implemented
- Basic dashboard
- EC2 instance created
- Bash installation script created with server configuration script also created

Checkpoint 2 — Notes and Search

- Notes database tables created and create/edit note forms created
- Note delete functionality

- Note page/editor working and notes are restricted to their owner
- Basic folders and categories with a search page

Checkpoint 3 — Fitness Tracking

- Exercise and workout model created with relationships
- Create workout form with edit/delete functionality
- Record sets, reps, and weight
- Workout history page
- Exercise-specific history filtering

Checkpoint 4 — Finance Tracking

- Financial accounts with create/edit/delete functionality
- Transaction model and categories with the same functionality
- Finance dashboard and account balance calculations
- Transaction filtering by category or date

Checkpoint 5 — Goals, Dashboard, Integration, and Visualization

- Goal creating form with edit/delete functionality
- Goal progress calculations
- Notes, fitness, and finance dashboard widgets
- Goal dashboard widget
- At least one fitness and one finance chart
- Improved Bootstrap styling

Checkpoint 6 — AWS EC2 and Deployment Automation

- Rails application deployed
- Nginx and Puma configured
- Production database configured
- Automatic application startup
- Application tested after EC2 restart
- Environment variables and secrets configured securely
- Global search implemented
- If I have extra time:
    - Active Storage configuration
    - Amazon S3 file storage
    - HTTPS configuration

Checkpoint 7 — Final Application and Testing

- All required functionality
- 7+ distinct pages
- Shared header and footer
- Navigation works and Bootstrap styling on all pages
- Create/update/delete forms all working
- Conditional search functionality
- User authorization
- Test application on EC2 and test EC2 startup automation
- Test all Bash scripts
- Bugs fixed
- Final UI and UX enhancements

- Potential additional features:
    - Drag-and-drop dashboard widgets
    - Note tags
    - S3 integration and File uploads
    - Recurring transactions
    - Workout templates

Testing Strategy

Automated testing will use the **Minitest** framework included with Ruby on Rails. Rails system tests using **Capybara** will also be used for important user workflows that involve multiple pages and forms.

Model tests will verify database validations, relationships, and calculations. Examples include verifying that notes belong to users, workouts correctly contain exercises and sets, financial transactions belong to accounts and categories, and goal progress calculations return the expected values.

Request/controller tests will verify that create, update, delete, search, and filtering operations return the correct responses. Authorization will also be tested to ensure that one authenticated user cannot view or modify another user's private notes, workouts, transactions, financial accounts, or goals.

System tests will cover important end-to-end workflows. These include registering and logging in, creating and editing a note, searching for a note, recording a workout, adding a financial transaction, and creating or updating a goal.

The complete automated test suite will be executable with `bin/rails test`, while Rails system tests can be run with `bin/rails test:system`. Tests will be run before major checkpoint submissions and before the final production deployment.

Install and Run Instructions

The project repository will contain three Bash scripts that automate local setup and AWS deployment: start.sh, deploy/setup-ec2.sh, and deploy/deploy.sh

start.sh

start.sh will allow the project to be started from a fresh clone in the supported Linux development environment. The script will check for and install required system packages, Ruby and Bundler dependencies, PostgreSQL dependencies, and any required frontend dependencies. It will then run bundle install, prepare the database using Rails database commands, and start the Rails development server.

A new developer should be able to clone the repository, make the script executable with chmod +x start.sh, and run ./start.sh. After the setup process finishes, the script will start the Rails application so that it can be accessed through the development server.

deploy/setup-ec2.sh

deploy/setup-ec2.sh will be run once when configuring a new Ubuntu-based AWS EC2 instance. It will update the operating system packages and install the software required by the production server, including Ruby, Bundler, PostgreSQL components, Git, Nginx, Puma dependencies, and other Rails build dependencies.

The script will create the required application directories, configure permissions, configure Nginx to forward HTTP traffic to Puma, and create a systemd service for the Nexus Rails application. The Rails service will be enabled with systemctl enable so that it automatically starts whenever the EC2 instance boots. Required production services such as Nginx will also be enabled at boot.

Secrets such as the Rails master key or production database credentials will not be committed to Git. They will be supplied to the server through protected environment variables or a server-side environment file.

deploy/deploy.sh

deploy/deploy.sh will be run from the developer's laptop whenever a new version of Nexus needs to be deployed. The script will connect to the EC2 instance using SSH and transfer or update the current application code. It will then run production dependency installation, database migrations or db:prepare, asset compilation when required, and restart the Nexus systemd service. This allows updated versions of the application to be deployed without manually repeating every server command.

Deployment Process

The deployment process will begin by creating an Ubuntu EC2 instance and configuring its security group to allow SSH and web traffic. The repository or setup script will then be copied to the new instance and deploy/setup-ec2.sh will be executed once.

After the server setup is complete, the local deployment configuration will be given the EC2 hostname, SSH username, and SSH key location. deploy/deploy.sh will then be executed from the developer's laptop to upload the application, install its production dependencies, prepare the production database, and start the application.

The deployment will be verified by visiting the EC2-hosted application in a browser. Finally, the EC2 instance will be rebooted and tested again. Because the Rails/Puma systemd service and Nginx are enabled at boot, the application should automatically return online without requiring a manual Rails command.

One caveat: your rubric references 07.02 for the exact responsibilities of these scripts, but that material wasn't included in the uploaded proposal. The section above satisfies the responsibilities stated in the rubric you gave me; if 07.02 imposes additional exact commands or behavior, those should also be incorporated.

Summary of Changes

I implemented all of the changes listed in the feedback for the original submission. I started by replacing the single image that contained all the wireframes, with individual images for each wireframe. This ensures proper visibility without needing to include a Google Drive link. I also moved the EC2 instance and Bash scripts from Checkpoint 6 to Checkpoint 1 so each module can be deployed as it gets built. Search is now a core feature and is found in the Functionality section. It was originally a stretch goal but a conditional search is required.

I also implemented the feedback given to me in the peer review. The two biggest issues that were noted was the absence of an automated testing section as well as install and run instructions. I implemented the automated testing section and identified the testing framework that will be used throughout the development of the project. I also included some of the various tests that will be used to ensure proper functionality. These include model tests, request/controller tests, and system tests. The install and run instructions were added in as well. This includes the three Bash scripts that will automate local setup and the deployment process was added as well.